

Arsitektur staging dan perubahan teknis setiap checkpoint

Dokumen ini untuk penilai dan pengembang. Isinya cara enam versi Kato Report berjalan bersamaan di satu domain staging, dan untuk setiap checkpoint: perubahan skema database, endpoint API, konfigurasi, serta keputusan desain beserta alasannya. Fitur dan cara mencobanya dijelaskan di panduan umum.

Situs: `ai-complaint-staging.yvrtz.workers.dev`

Repo: `rdnsa/AI-complaint`, branch `staging`

Fitur dan skenario uji: `panduan umum`

Daftar isi

1. Ringkasan
2. Rencana sprint
3. Arsitektur staging
4. Sprint 1 / Checkpoint 1: Pondasi
5. Sprint 2 / Checkpoint 2: Sesuai kampus
6. Sprint 3 / Checkpoint 3: Bukti, jejak, dan akun
7. Sprint 4 / Checkpoint 4: Rapi dan tajam
8. Sprint 5 / Checkpoint 5: AI kedua
9. Sprint 6 / Checkpoint 6: Siap pakai
10. Rekap perkembangan
11. Operasional staging
12. Lampiran: daftar commit

1. Ringkasan

Sistem yang dibangun, dan bagaimana riwayatnya dipecah menjadi checkpoint.

Masalah. Keluhan tentang toilet kampus jarang sampai ke petugas kebersihan. Mahasiswa tidak tahu harus melapor ke siapa, dan tidak ada cara untuk memastikan keluhannya ditindaklanjuti. Akibatnya masalah kecil dibiarkan sampai menjadi besar.

Solusi. Stiker QR di setiap lantai. Mahasiswa memindai QR dan menulis keluhan dengan bahasa sehari-hari. AI (DeepSeek) menggolongkan keluhan, menentukan prioritas, dan menyusun perintah kerja. Petugas menutup laporan dengan foto bukti yang diperiksa AI kedua (Google Gemini). Supervisor memantau semuanya lewat dashboard dan Chatbot AI.

Cara melaporkan progres. Sistem final ada di branch `main`. Riwayat pengembangannya dibagi menjadi enam checkpoint, masing-masing ditandai dengan tag git `checkpoint-1` sampai `checkpoint-6`. Setiap checkpoint dijalankan sebagai aplikasi tersendiri di situs staging, sehingga siapa pun bisa mencoba sistem persis seperti pada tahap tersebut.

#	CHECKPOINT	INTI PERUBAHAN	COMMIT	ENDPOINT API
1	Pondasi	Lapor via QR, klasifikasi dan prioritas dengan LLM, dashboard petugas, ringkasan harian	3	16
2	Sesuai kampus	Gedung UPI, QR per lantai, dwibahasa, pelacakan status, papan laporan publik	4	19
3	Bukti, jejak, akun	Foto wajib, bukti penyelesaian, log aktivitas, grafik, akun berperan, papan peringkat	2	26
4	Rapi dan tajam	Arsitektur berlapis, dashboard analitis, "Laporan saya" per akun, dokumentasi	4	26
5	AI kedua	Verifikasi foto dengan vision AI, Chatbot AI, mode gelap	5	28
6	Siap pakai	Tiga peran, petugas tanpa login, kamera langsung, laporan pekerjaan, rebrand Kato Report, failover API key	17	32

2. Rencana sprint

Pengembangan dibagi menjadi enam sprint. Setiap sprint diakhiri satu checkpoint yang bisa dicoba di staging, jadi Sprint N sama dengan Checkpoint N.

SPRINT	TUJUAN	FOKUS TEKNIS
Sprint 1	Membuktikan alur inti: keluhan dari QR sampai menjadi tiket kerja yang bisa langsung dikerjakan petugas.	Worker + D1 + R2, pipeline LLM asinkron, validasi keluaran AI
Sprint 2	Menyesuaikan sistem dengan kampus yang sebenarnya, dan membuat penanganan laporan terlihat oleh semua orang.	Skema lokasi per gedung dan lantai, seed idempoten, endpoint publik yang dibatasi
Sprint 3	Membuat penyelesaian bisa dibuktikan, dan setiap tindakan bisa dipertanggungjawabkan per orang.	Aturan bukti di server, tabel audit tanpa FK, hashing sandi PBKDF2
Sprint 4	Merapikan fondasi kode dan mempertajam dashboard sebelum menambah fitur AI berikutnya.	Refaktor ke lima lapisan tanpa mengubah perilaku, grafik yang menjawab pertanyaan
Sprint 5	Memakai AI untuk memverifikasi pekerjaan petugas dan menjawab pertanyaan tentang data.	Model vision kedua, tool calling dengan kuota, kolom hasil verifikasi
Sprint 6	Menyesuaikan alur dengan cara kerja di lapangan, dan menyiapkan sistem untuk dipakai sungguhan.	Skema berbahasa Inggris, peran supervisor, frontend per peran, ketahanan API key

Rincian perubahan teknis dari sprint ke sprint ada di awal bagian setiap checkpoint.

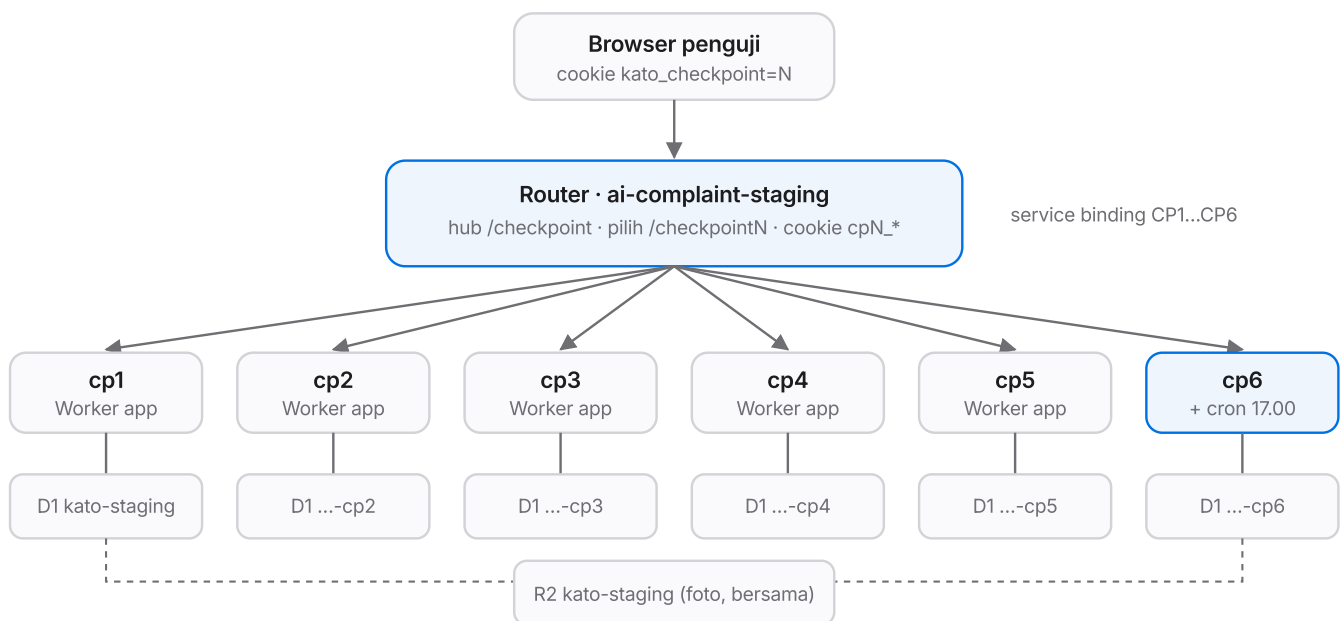
3. Arsitektur staging

Bagaimana enam versi aplikasi berjalan bersamaan di satu domain tanpa mengubah kode aplikasinya.

Model branch dan tag

- `main` berisi sistem final, dan dari branch inilah produksi di-deploy.
- **Tag** `checkpoint-1` **sampai** `checkpoint-6` menandai commit di `main` yang menjadi batas setiap checkpoint: `dd9c91a` , `feb7e49` , `2e18c48` , `d2e7d72` , `e44958b` , dan `defb699` .
- `staging` dimulai dari `checkpoint-1` . Setiap kali melapor, tag berikutnya di-merge dengan `git merge --no-ff checkpoint-N` . Arahnya selalu satu: `main` ke `staging` , tidak pernah sebaliknya.
- File khusus staging tidak pernah ada di `main` , sehingga merge tidak pernah konflik. File-file itu adalah `STAGING.md` , `staging.config.json` , `scripts/staging.mjs` , dan folder `staging/` (`router.mjs` , `checkpoints.mjs` , `journey.mjs` , `docs/`). Setelah checkpoint 6, isi `staging` sama dengan `main` ditambah file-file ini.

Topologi



Semua permintaan masuk ke router, lalu diteruskan ke Worker checkpoint yang dipilih. Worker checkpoint tidak punya alamat publik sendiri.

Router (`staging/router.mjs`)

Aplikasi di setiap checkpoint memakai alamat absolut seperti `/api/...` dan `/lapor/...` , sehingga tidak bisa dipasang di bawah awalan seperti `/checkpoint3/` tanpa mengubah kode lamanya. Router menyelesaikan masalah itu tanpa menyentuh kode aplikasi:

1. `/checkpoint` menyajikan halaman hub statis dari binding `ASSETS` .
2. `/checkpointN[/jalur]` memasang cookie `kato_checkpoint=N` (30 hari, `SameSite=Lax` , `Secure`) lalu mengalihkan ke `/jalur` . Checkpoint yang belum dilaporkan dialihkan ke hub.

- Permintaan lain diteruskan ke Worker `ai-complaint-staging-cpN` lewat *service binding* `CPn`, sesuai cookie. Kalau cookie tidak ada, yang dipakai checkpoint terbaru. URL permintaan tidak diubah, jadi aplikasi melihat domain staging sebagai domainnya sendiri.
- Cookie aplikasi dipisah per checkpoint.** Setiap `Set-Cookie` dari aplikasi disimpan dengan awalan `cpN_`, misalnya `cp1_sesi_petugas`. Saat meneruskan permintaan, router hanya mengirim cookie berawalan checkpoint aktif dan mengembalikan namanya ke nama aslinya. Tanpa pemisahan ini, sesi dari satu checkpoint bisa diterima checkpoint lain yang memakai `AUTH_SECRET` sama, padahal id pengguna yang sama di database lain bisa berarti orang lain.
- Respons aplikasi diteruskan apa adanya; router tidak menyisipkan apa pun ke halaman, sehingga tampilan setiap checkpoint sama persis dengan aslinya.

Worker, database, dan penyimpanan

SUMBER DAYA	NAMA	KETERANGAN
Worker router	<code>ai-complaint-staging</code>	Alamat publik (<code>workers.dev</code> , atau domain sendiri lewat <code>staging.config.json</code>). Tidak punya database.
Worker aplikasi	<code>ai-complaint-staging-cp1 ... -cp6</code>	Kode persis seperti di checkpoint masing-masing. <code>workers_dev</code> dan <code>preview_urls</code> dimatikan, jadi hanya bisa dicapai lewat router.
Database D1	<code>kato-staging (cp1), kato-staging-cp2 ... -cp6</code>	Satu per checkpoint, karena skemanya berubah (misalnya checkpoint 6 mengganti seluruh skema ke bahasa Inggris). Migrasi dijalankan berurutan sampai migrasi milik checkpoint itu.
Bucket R2	<code>kato-staging</code>	Dipakai bersama. Kunci objek foto berupa UUID acak, jadi tidak pernah bentrok.
Cron	hanya di checkpoint terbaru	Paket gratis Cloudflare membatasi 5 cron per akun. Cron checkpoint sebelumnya dilepas dengan <code>wrangler triggers deploy</code> .

Skrip deploy (`scripts/staging.mjs`)

Skrip ini tidak pernah mengubah `wrangler.jsonc` milik `main`. Sebagai gantinya, skrip membaca file itu dan menulis konfigurasi sementara `wrangler.staging.jsonc` yang dihapus lagi setelah selesai. Urutan kerja `node scripts/staging.mjs` :

- Menentukan checkpoint terbaru, yaitu tag terakhir yang commit-nya sudah menjadi leluhur `HEAD` (`git merge-base --is-ancestor`).

- 2 Mencari database D1 checkpoint itu, atau membuatnya (`wrangler d1 create --update-config=false`) kalau belum ada.
- 3 Menjalankan migrasi. Untuk database baru, atau dengan argumen `seed` , skrip juga mengisi data toilet dari `seed/toilets.sql` .
- 4 Men-deploy Worker checkpoint dengan nama, database, dan bucket staging, lalu mengirim secret dari `.dev.vars` .
- 5 Melepas cron dari Worker checkpoint sebelumnya.
- 6 Membuat halaman hub dan dokumentasi (`staging/journey.mjs`), lalu men-deploy router dengan service binding ke semua checkpoint yang sudah dilaporkan.

Secret

Secret Cloudflare tidak bisa dibaca kembali, jadi nilainya diambil dari `.dev.vars` lokal, yang tidak ikut di-commit. Secret yang dipakai adalah `LLM_API_KEY` , `VISION_API_KEY` , `AUTH_SECRET` , dan `PETUGAS_PASSWORD` (khusus checkpoint 1–2). Sejak checkpoint 6, `LLM_API_KEY` dan `VISION_API_KEY` boleh berisi beberapa key yang dipisah koma. Kode checkpoint yang lebih lama akan membaca daftar itu sebagai satu key yang tidak valid, jadi skrip memeriksa kodenya dulu: kalau fungsi `splitKeys` belum ada, hanya key pertama yang dikirim.



Pondasi: keluhan menjadi tiket kerja

3

commit

35

file berubah

+7.148

baris

16

endpoint API

Fitur dan skenario uji checkpoint ini: panduan umum.

Sprint 1

Tujuan: Membuktikan alur inti: keluhan dari QR sampai menjadi tiket kerja yang bisa langsung dikerjakan petugas. **Fokus teknis:** Worker + D1 + R2, pipeline LLM asinkron, validasi keluaran AI.

Kondisi awal → Sprint 1 (teknis)

ASPEK	SEBELUM SPRINT 1	SPRINT 1
Infrastruktur	Repositori kosong	Satu Cloudflare Worker (Hono) untuk API dan frontend React; D1 untuk data, R2 untuk foto
Database	–	Migrasi <code>0001</code> : <code>toilets</code> , <code>reports</code> , <code>daily_summaries</code>
API	–	16 endpoint
AI	–	DeepSeek lewat <code>ctx.waitUntil()</code> ; keluaran divalidasi Zod dan dipaksa ke enum; bisa dianalisis ulang
Autentikasi	–	Satu sandi bersama (<code>PETUGAS_PASSWORD</code>) dan cookie sesi bertanda tangan <code>AUTH_SECRET</code>
Deploy	–	<code>build.command</code> membangun frontend otomatis; cron <code>0 10 * * *</code> untuk ringkasan harian

Fondasi

- Stack:** TypeScript di Cloudflare Workers dengan Hono. Database D1 (SQLite), penyimpanan foto R2, dan frontend React + Vite + Tailwind. Frontend dan API ada di **satu Worker**, sehingga tidak perlu CORS dan cukup sekali deploy. Aset statis disajikan lewat binding `ASSETS` dengan `run_worker_first: ["/api/*"]` dan fallback SPA.

- **Analisis asinkron:** `POST /api/reports` langsung menyimpan laporan dan menjawab. Analisis LLM dijalankan lewat `ctx.waitUntil()` setelah respons terkirim, sementara halaman konfirmasi melakukan polling.
- **Ketahanan:** kalau LLM gagal, laporan ditandai `ai_status = 'gagal'` beserta pesan errornya. Laporan itu tetap tampil dan bisa dianalisis ulang lewat `POST /api/reports/:id/analisa-ulang`. Latensi setiap panggilan dicatat di `ai_ms`.
- **Validasi keluaran LLM:** JSON dari DeepSeek (`response_format: json_object` , temperatur rendah, contoh *few-shot*) divalidasi dengan Zod, lalu dipaksa masuk ke enum yang dikenal database. Kategori asing dibuang, dan prioritas yang tidak dikenal menjadi *sedang*.
- **Foto lewat Worker:** `GET /api/uploads/:key` membaca objek langsung dari R2, dibatasi hanya untuk prefiks `laporan/` , dan meneruskan ETag. Alasannya, sebagian ISP di Indonesia membajak DNS untuk `*.r2.dev` , sehingga foto gagal dimuat di jaringan kampus.
- **Build otomatis:** `build.command` di `wrangler.jsonc` membangun frontend sebelum deploy, supaya `wrangler deploy` berjalan sendiri, termasuk di Workers Builds.
- **Cron** `0 10 * * *` (17.00 WIB) menyusun `daily_summaries` . Ringkasan juga bisa dibuat manual lewat `POST /api/summary/generate` .

Database: migrasi `0001_init.sql`

Tabel `toilets` (satu baris per WC fisik, `id` seperti `A-2-PRIA` di-encode ke QR), `reports` (keluhan, status, hasil AI: kategori, prioritas, ringkasan, rekomendasi, `ai_status`), dan `daily_summaries` , ditambah empat indeks untuk urutan dashboard.

Endpoint API (16)

<code>GET /api/health</code>	<code>GET /api/reports/:id</code>
<code>POST /api/auth/login</code>	<code>PATCH /api/reports/:id</code>
<code>POST /api/auth/logout</code>	<code>POST /api/reports/:id/analisa-ulang</code>
<code>GET /api/auth/me</code>	<code>POST /api/uploads</code>
<code>GET /api/toilets</code>	<code>GET /api/uploads/:key</code>
<code>GET /api/toilets/:id</code>	<code>GET /api/summary</code>
<code>POST /api/reports</code>	<code>GET /api/summary/stats</code>
<code>GET /api/reports</code>	<code>POST /api/summary/generate</code>

Halaman: `/` , `/lapor/:toiletId` , `/laporan/:id` , `/petugas` . **Konfigurasi:** vars `LLM_BASE_URL` dan `LLM_MODEL` (`deepseek-chat`); secret `LLM_API_KEY` , `AUTH_SECRET` (tanda tangan cookie sesi), dan `PETUGAS_PASSWORD` .

Sesuai kampus, dan bisa dilacak

4

commit

27

file berubah

+1.914 /

-514

baris

19

endpoint API

Fitur dan skenario uji checkpoint ini: panduan umum.

Sprint 2

Tujuan: Menyesuaikan sistem dengan kampus yang sebenarnya, dan membuat penanganan laporan terlihat oleh semua orang. **Fokus teknis:** Skema lokasi per gedung dan lantai, seed idempoten, endpoint publik yang dibatasi.

Yang berubah dari Sprint 1 ke Sprint 2 (teknis)

ASPEK	SPRINT 1	SPRINT 2
Model lokasi	<code>toilets</code> satu baris per WC dengan nama gedung sebagai teks	Tabel <code>gedung</code> + <code>toilets.gedung_kode</code> ; id QR <code><gedung>-<lantai></code> (migrasi <code>0002</code>)
Seed data toilet	Menghapus isi tabel, sehingga gagal kena <i>foreign key</i> begitu ada laporan	Menonaktifkan semua lalu mengaktifkan yang terdaftar; aman dijalankan ulang
API	16 endpoint	19 endpoint: + <code>/api/lokasi</code> , <code>/api/lokasi/:id</code> , <code>/api/reports/publik</code> , <code>/api/reports/ringkas</code> , <code>DELETE /api/reports/:id</code> ; - <code>/api/toilets</code>
Pelacakan	–	Id laporan di <code>localStorage</code> , status diambil sekaligus lewat satu permintaan
i18n	–	Kamus sendiri tanpa pustaka tambahan

- **Migrasi** `0002_gedung_upi.sql` : tabel `gedung` berisi sepuluh gedung, dan `toilets` mendapat `gedung_kode` . Kolom `gedung` dan `nama` dihapus, dan ditambah indeks lokasi. Id lokasi QR berformat `<kode gedung>-<lantai>` , misalnya `A-1` .

- **Seed yang aman dijalankan ulang:** semua WC dinonaktifkan dulu, lalu yang terdaftar diaktifkan kembali. Versi sebelumnya menghapus isi tabel dan gagal karena *foreign key* begitu ada laporan. Lantai yang terdaftar: A 1–3, B 1, C 1–5, G 1, H 1, dan I 1–2. Gedung D, E, F, dan J disiapkan sebagai komentar.
- **Pelacakan tetap anonim:** id laporan (UUID acak) disimpan di `localStorage`. `GET /api/reports/ringkas` mengambil status banyak laporan dalam satu permintaan. Halaman konfirmasi menyegarkan diri dengan cepat selama analisis berjalan, melambat setelahnya, dan berhenti saat tab tidak terlihat.
- **Papan publik** `GET /api/reports/publik` sengaja tidak menyertakan teks asli, foto pelapor, maupun nama petugas. Mengubah, menganalisis ulang, dan menghapus tetap membutuhkan sesi petugas.
- `DELETE /api/reports/:id` juga menghapus foto di R2 agar tidak meninggalkan berkas yatim.
- **i18n** memakai kamus sendiri tanpa pustaka. Hasil LLM tetap berbahasa Indonesia karena pembacanya petugas.

+ **endpoint:** `GET /api/lokasi`, `GET /api/lokasi/:id`, `GET /api/reports/publik`, `GET /api/reports/ringkas`, `DELETE /api/reports/:id`. – **endpoint:** `GET /api/toilets`, `GET /api/toilets/:id`.

Halaman baru: `/laporan`, dan `/lapor/:lokasiId` menggantikan `/lapor/:toiletId`.

Bukti, jejak, dan akun

2

commit

35

file berubah

+2.128 /

-178

baris

26

endpoint API

Fitur dan skenario uji checkpoint ini: panduan umum.

Sprint 3

Tujuan: Membuat penyelesaian bisa dibuktikan, dan setiap tindakan bisa dipertanggungjawabkan per orang. **Fokus teknis:** Aturan bukti di server, tabel audit tanpa FK, hashing sandi PBKDF2.

Yang berubah dari Sprint 2 ke Sprint 3 (teknis)

ASPEK	SPRINT 2	SPRINT 3
Database	3 tabel inti + <code>gedung</code>	+ <code>reports.foto_selesai_key</code> , tabel <code>aktivitas</code> tanpa FK (<code>0003</code>); + tabel <code>pengguna</code> , <code>reports.pelapor_id</code> , admin bawaan (<code>0004</code>)
Autentikasi	Sandi bersama dibandingkan dengan secret	Akun per orang, sandi di-hash PBKDF2-SHA256 100.000 iterasi; secret <code>PETUGAS_PASSWORD</code> dihapus
Aturan bukti	–	Dijaga di server, bukan hanya disembunyikan di antarmuka
API	19 endpoint	26 endpoint: + <code>/api/auth/masuk daftar keluar saya</code> , <code>/api/summary/grafik</code> , <code>/api/aktivitas</code> , <code>/api/pengguna</code> , <code>/api/peringkat</code> ; – <code>/api/auth/login logout me</code>

- **Migrasi** `0003_bukti_dan_aktivitas.sql` : `reports.foto_selesai_key` dan tabel `aktivitas` dengan tiga indeks. Tabel `aktivitas` sengaja **tanpa foreign key** ke `reports`, karena justru laporan yang dihapus yang paling perlu terlacak. Isinya disalin ke catatan sebelum baris aslinya dihapus.
- **Migrasi** `0004_pengguna.sql` : tabel `pengguna` (peran `admin` / `petugas` / `pelapor`), `reports.pelapor_id`, dan akun admin bawaan.

- **Aturan wajib foto dijaga di server**, bukan hanya disembunyikan di antarmuka, sehingga tidak bisa diakali lewat klien lain.
- **Kata sandi** di-hash dengan PBKDF2-SHA256, 100.000 iterasi, salt per pengguna, lewat WebCrypto. Bcrypt dan argon2 tidak tersedia di runtime Workers tanpa WebAssembly tambahan. Pesan galat untuk "akun tidak ada" dan "sandi salah" dibuat identik untuk mencegah enumerasi username. Admin tidak bisa menonaktifkan akunnya sendiri. Secret `PETUGAS_PASSWORD` tidak dipakai lagi.
- **Palet grafik** dipilih lewat validator: rentang terang, ambang chroma, keterpisahan untuk buta warna, dan kontras terhadap latar.

+ **endpoint:** `POST /api/auth/masuk` , `/daftar` , `/keluar` , `GET /api/auth/saya` , `GET /api/summary/grafik` , `GET /api/aktivitas` , `GET/POST /api/pengguna` , `PATCH /api/pengguna/:id` , `GET /api/peringkat` . – **endpoint:** `/api/auth/login` , `/logout` , `/me` .

Halaman baru: `/masuk` , `/daftar` , `/peringkat` .

Rapi di dalam, tajam di dashboard

4

commit

72

file berubah

+8.574 /

-1.468

baris (termasuk
dokumen)

26

endpoint API

Fitur dan skenario uji checkpoint ini: panduan umum.

Sprint 4

Tujuan: Merapikan fondasi kode dan mempertajam dashboard sebelum menambah fitur AI berikutnya. **Fokus teknis:** Refaktor ke lima lapisan tanpa mengubah perilaku, grafik yang menjawab pertanyaan.

Yang berubah dari Sprint 3 ke Sprint 4 (teknis)

ASPEK	SPRINT 3	SPRINT 4
Struktur backend	37 dari 43 pernyataan SQL berada di <i>route handler</i>	Lima lapisan: <code>domain/</code> , <code>repositories/</code> , <code>services/</code> , <code>adapters/</code> , <code>routes/</code>
Aturan bukti	Dicek di masing-masing route	Satu aturan di lapisan domain
API	26 endpoint	26 endpoint: + <code>/api/reports/saya</code> ; - <code>/api/reports/ringkas</code>
Database	–	Tidak berubah (tidak ada migrasi)
Bahasa kode	Komentar berbahasa Indonesia	README ditulis ulang, dan semua komentar berbahasa Inggris

- **Arsitektur berlapis.** Sebelumnya 37 dari 43 pernyataan SQL berada di dalam *route handler*. Sekarang ada lima lapisan dengan ketergantungan yang hanya mengarah ke dalam:
 - `domain/` : kosakata dan aturan bisnis, tanpa import Hono, D1, maupun R2
 - `repositories/` : semua SQL, satu file per agregat
 - `services/` : *use case*, tanpa Hono
 - `adapters/` : LLM, R2, hashing, sesi, dan jam
 - `routes/` : parsing, validasi, delegasi, dan format

`index.ts` menjadi satu-satunya *composition root*.

- **Aturan "tutup hanya dengan bukti"** dipindah ke lapisan domain, sehingga semua pemanggil memutuskannya dengan cara yang sama.
- **Tidak ada migrasi.** Tidak ada jalur API, kolom database, atau perilaku yang berubah, kecuali "Laporan saya": `GET /api/reports/saya` menggantikan `GET /api/reports/ringkas`. Pelapor anonim tetap bisa melacak lewat tautan konfirmasi.
- **Grafik:** skala sekuensial peta panas diperiksa agar luminansnya menurun monoton (0,933 ke 0,164). Nilai nol memakai warna permukaan, supaya "tidak ada" tidak tertukar dengan "sedikit". Setiap segmen bertumpuk diberi celah 2 px.
- **Bahasa:** README ditulis ulang, dan semua komentar kode diterjemahkan ke bahasa Inggris tanpa mengubah logika.
- **Pengujian ulang setelah restrukturisasi:** analisis LLM, wajib bukti, papan publik, papan peringkat, grafik, statistik, jejak audit, ringkasan harian, dan batas wewenang ketiga peran.

AI kedua: memeriksa foto dan menjawab pertanyaan

5

commit

43

file berubah

+3.128 / -330

baris

28

endpoint API

Fitur dan skenario uji checkpoint ini: panduan umum.

Sprint 5

Tujuan: Memakai AI untuk memverifikasi pekerjaan petugas dan menjawab pertanyaan tentang data. **Fokus teknis:** Model vision kedua, tool calling dengan kuota, kolom hasil verifikasi.

Yang berubah dari Sprint 4 ke Sprint 5 (teknis)

ASPEK	SPRINT 4	SPRINT 5
Model AI	Satu model teks (DeepSeek)	Dua model: + model vision (Gemini) lewat endpoint yang kompatibel dengan OpenAI
Database	–	+ bukti_ai_hasil , bukti_ai_alasan , bukti_ai_model , bukti_ai_ms (0005)
API	26 endpoint	28 endpoint: + GET/POST /api/tanya (tool calling, 50 pertanyaan/hari, 20 per IP)
Konfigurasi	LLM_* saja	+ VISION_BASE_URL , VISION_MODEL , VISION_API_KEY
Anggaran token AI	Lebih kecil	Dinaikkan ke 2.000 agar model <i>reasoning</i> tidak terpotong

- Dua model AI.** DeepSeek tidak bisa melihat gambar, jadi foto dinilai oleh model vision terpisah lewat endpoint yang kompatibel dengan OpenAI: VISION_BASE_URL Gemini dengan VISION_MODEL = gemini-3.6-flash . VISION_API_KEY jatuh ke LLM_API_KEY kalau tidak diisi.
- Migrasi** 0005_verifikasi_bukti.sql : kolom bukti_ai_hasil (bersih | kotor | bukan_toilet), bukti_ai_alasan , bukti_ai_model , dan bukti_ai_ms . Status selesai hanya diberikan kalau putusannya

bersih .

- **Chatbot** (GET/POST /api/tanya) memakai *tool calling*. Model tidak pernah melihat database, hanya definisi alat: hitung laporan, waktu penyelesaian, daftar laporan, dan ringkasan harian. Hasil setiap alat berupa agregat kecil, sehingga biaya per pertanyaan tetap kecil berapa pun jumlah laporannya. Loop dibatasi beberapa putaran, lalu model dipaksa menjawab. Kuotanya 50 pertanyaan per hari untuk seluruh kampus dan 20 per IP tanpa akun. Nama petugas hanya diberikan kepada pengguna yang login.
- **Anggaran token LLM** dinaikkan ke 2.000 agar model *reasoning* tidak terpotong.
- **Tema** memakai konteks React dengan kelas `.dark` dan pilihan yang diingat di browser.

+ **endpoint:** GET /api/tanya , POST /api/tanya . **Konfigurasi baru:** vars `VISION_BASE_URL` dan `VISION_MODEL` , secret `VISION_API_KEY` .

Dirancang ulang untuk pemakaian sehari-hari

17

commit

171

file berubah

+15.332 /

-6.211

baris

32

endpoint API

Fitur dan skenario uji checkpoint ini: panduan umum.

Sprint 6

Tujuan: Menyesuaikan alur dengan cara kerja di lapangan, dan menyiapkan sistem untuk dipakai sungguhan. **Fokus teknis:** Skema berbahasa Inggris, peran supervisor, frontend per peran, ketahanan API key.

Yang berubah dari Sprint 5 ke Sprint 6 (teknis)

ASPEK	SPRINT 5	SPRINT 6
Database	Skema berbahasa Indonesia, peran <code>admin</code> / <code>petugas</code> / <code>pelapor</code>	+ tabel <code>pekerjaan</code> (<code>0006</code>); <code>admin</code> menjadi <code>spv</code> dan <code>petugas</code> tanpa sandi (<code>0007</code>); seluruh skema ke bahasa Inggris (<code>0008</code>)
API	28 endpoint berbahasa Indonesia	32 endpoint berbahasa Inggris: + <code>/api/work-logs</code> , <code>/api/staff</code> , <code>/api/reports/open</code>
Frontend	Halaman umum: beranda, lapor, dashboard	Halaman per peran (<code>StudentHome</code> , <code>StaffHome</code> , <code>StaffFloor</code> , <code>SupervisorDashboard</code>) + komponen <code>Camera</code> , <code>StaffPicker</code> , <code>RoleSwitch</code> , <code>TimeFilter</code> , peta interaktif, <code>ZoomableImage</code>
Desain	Palet UPI	Token warna di variabel CSS (terang dan gelap), gaya halaman produk Apple
API key AI	Satu key per secret	Beberapa key dipisah koma; respons 429/401/403 dicoba ulang dengan key berikutnya
Dokumen	Panduan umum dan dokumentasi teknis	+ BPMN <i>to-be</i> , diagram arsitektur, penjelasan diagram, dan PRD

- **Migrasi** `0006_laporan_pekerjaan.sql` : tabel `pekerjaan` untuk laporan pekerjaan petugas, diindeks per petugas, toilet, dan waktu.
- **Migrasi** `0007_peran_spv.sql` : peran menjadi `spv / petugas / pelapor`, dan `admin` diubah menjadi `spv`. Baris petugas kini cukup berisi nama: username dan sandi menjadi opsional (`UNIQUE` di SQLite mengizinkan banyak NULL). Tabel dibangun ulang lewat salinan karena SQLite tidak bisa mengubah `CHECK`.
- **Migrasi** `0008_english_schema.sql` : seluruh skema diganti ke bahasa Inggris dengan pola *create new, copy, rename*, tanpa kehilangan data. Peran menjadi `supervisor / staff / reporter`, dan tabel aktivitas menjadi `activity_log`.
- **Frontend baru**: halaman `StudentHome`, `StaffHome`, `StaffFloor`, dan `SupervisorDashboard`. Komponen baru antara lain `Camera` (kamera langsung), `StaffPicker` (pilihan nama disimpan di `localStorage`), `RoleSwitch`, `WorkLogPanel`, `TimeFilter`, `ZoomableImage`, `ImageViewer`, `QrCodesPanel`, dan peta interaktif. Alamat lama (`/lapor`, `/petugas`, `/laporan`, `/masuk`, `/spv`, dan lainnya) dialihkan lewat `LegacyRedirect`.
- **API diganti ke bahasa Inggris** dan diperluas dengan `/api/work-logs`, `/api/staff`, `/api/reports/open`, dan `/api/reports/mine`.
- **Desain**: token warna dibaca dari variabel CSS (terang di `:root`, gelap di `.dark`), dengan gaya ala halaman produk Apple dan mode terang sebagai bawaan.
- **Failover API key**: `LLM_API_KEY` dan `VISION_API_KEY` boleh berisi beberapa key yang dipisah koma. Respons 429 (limit) atau 401/403 (key ditolak) dicoba ulang dengan key berikutnya, dan error terakhir dilaporkan kalau semua key gagal.
- **Dokumen**: BPMN *to-be*, diagram arsitektur, penjelasan diagram, dan PRD, dengan skrip `npm run docs` untuk membuat PDF-nya.

+ **endpoint (nama baru/baru)**: `/api/auth/login|register|logout|me`, `/api/locations[:id]`, `/api/reports/public|open|mine`, `/api/reports/:id/reanalyze`, `GET/POST /api/work-logs`, `GET /api/staff`, `/api/summary/charts`, `/api/activity`, `/api/users[:id]`, `/api/leaderboard`, `GET/POST /api/ask`. Padanan berbahasa Indonesianya dari checkpoint 3–5 dihapus.

4. Rekap perkembangan

Ukuran perubahan di setiap checkpoint.

Statistik per checkpoint

CHECKPOINT	COMMIT	FILE BERUBAH	BARIS +	BARIS -	ENDPOINT	MIGRASI
1 Pondasi	3	35	7.148	1	16	0001
2 Sesuai kampus	4	27	1.914	514	19	0002
3 Bukti, jejak, akun	2	35	2.128	178	26	0003, 0004
4 Rapi dan tajam	4	72	8.574	1.468	26	-
5 AI kedua	5	43	3.128	330	28	0005
6 Siap pakai	17	171	15.332	6.211	32	0006, 0007, 0008

Angka diambil dari `git diff --shortstat` antar tag. Angka checkpoint 4 dan 6 termasuk dokumen HTML/PDF yang ikut di-commit.

5. Operasional staging

Untuk pengembang yang merawat situs staging.

Persiapan sekali saja

```
npx wrangler r2 bucket create kato-staging
npx wrangler d1 create kato-staging --update-config=false # isi database_id di staging.config.json
cp .dev.vars.example .dev.vars # isi LLM_API_KEY, VISION_API_KEY, AUTH_SECRET, P
```

staging.config.json berisi nama Worker router, nama dan id database checkpoint 1, nama bucket, dan domain (opsional). Kalau domain diisi, domainnya harus berupa zone di akun Cloudflare yang sama.

Naik ke checkpoint berikutnya

```
git switch staging
git merge --no-ff checkpoint-N -m "Checkpoint N"
node scripts/staging.mjs # database dibuat dan diisi otomatis bila baru
git push origin staging
```

Perintah lain

PERINTAH	FUNGSI
node scripts/staging.mjs seed	Deploy, sekaligus mengisi ulang data toilet di database checkpoint terbaru (aman dijalankan berulang)
node scripts/staging.mjs secrets	Mengirim ulang secret dari .dev.vars ke Worker checkpoint terbaru
node scripts/staging.mjs config	Hanya menulis wrangler.staging.jsonc untuk diperiksa

Masalah yang pernah terjadi

GEJALA	PENYEBAB	PENANGANAN
Sesaat setelah deploy, checkpoint baru menjawab "Endpoint tidak ditemukan" atau menampilkan checkpoint sebelumnya	Router versi baru belum menyebar ke semua lokasi Cloudflare, jadi sebagian permintaan masih memakai daftar checkpoint lama	Tunggu sekitar satu menit
<code>d1 execute</code> gagal dengan <code>InternalError ... Please try again</code>	Gangguan sementara di Cloudflare saat mengimpor file SQL	Jalankan <code>node scripts/staging.mjs seed</code> lagi
<code>wrangler d1 create</code> menambahkan binding ke <code>wrangler.jsonc</code>	Wrangler menawarkan mengubah konfigurasi	Selalu pakai <code>--update-config=false</code> , dan jangan commit perubahan itu ke <code>main</code>

Batas paket gratis Cloudflare

- **D1:** 10 database per akun. Staging memakai 6, dan akun ini sudah terisi 10.
- **Cron:** 5 per akun. Staging hanya memakai 1, di checkpoint terbaru.

Membongkar staging

```
npx wrangler delete --name ai-complaint-staging
for n in 1 2 3 4 5 6; do npx wrangler delete --name ai-complaint-staging-cp$n; done
npx wrangler d1 delete kato-staging
for n in 2 3 4 5 6; do npx wrangler d1 delete kato-staging-cp$n; done
npx wrangler r2 bucket delete kato-staging    # kosongkan isinya dulu
```

Hati-hati: jangan pernah menjalankan perintah di atas dengan nama `ai-complaint` atau `kato`. Keduanya milik produksi.

6. Lampiran: daftar commit per checkpoint

Dari `git log --reverse checkpoint-(N-1)..checkpoint-N`. Commit awal `949c5f7` ("first commit") tidak dihitung.

Checkpoint 1

227dcbc Bangun sistem klasifikasi dan prioritas keluhan toilet berbasis LLM
28be726 Bangun frontend otomatis sebelum deploy
dd9c91a Sajikan foto lampiran lewat Worker, bukan domain r2.dev

Checkpoint 2

e1b3743 Terapkan identitas UPI Tasikmalaya, QR per lantai, dan dwibahasa
68a1e28 Sesuaikan daftar lokasi dengan kondisi gedung sebenarnya
37a15cc Beri pelapor cara melacak status laporannya
feb7e49 Buka daftar laporan untuk umum, tanpa membuka wewenang mengubahnya

Checkpoint 3

88ffa4a Wajibkan bukti foto, catat aktivitas, dan tampilkan grafik
2e18c48 Ganti sandi bersama dengan akun berperan, dan tambah papan peringkat

Checkpoint 4

f7d5727 Rewrite the README in English and translate all code comments
603e56f Restructure the backend into layers, and move "My reports" onto the account
c1f5bc4 Deepen the dashboard charts from counts into questions
d2e7d72 Add two documents describing the system, for two audiences

Checkpoint 5

9b2594d feat: integrate vision AI for automated verification of proof-of-completion photos
c21d18b chore: upgrade default vision model to Gemini 3.6-flash
76e303b chore: increase LLM response budget to 2000 tokens to accommodate reasoning models
7673129 docs: update documentation and generated PDFs with expanded guides and AI photo validation details
e44958b feat: add AI-powered Q&A feature for querying report analytics

Checkpoint 6

777b25c feat: add camera component for photo capture and implement staff work panel
5abc63b feat: implement time filter component and related functionality for reports and work logs
298fa53 feat: enhance header with dynamic supervisor link and accessibility features
7341294 feat: add staff and student home pages, supervisor dashboard, and staff floor functionality
39f573e refactor: update staff, student, and supervisor dashboard pages for improved readability and functionality
365a461 Refactor documentation and UI components for QR code and Chatbot AI features
ff63471 feat: add MapViewer component for interactive campus map with zoom and pan functionality

f9e4aeb feat: add MapViewer component for interactive campus map with zoom and drag functionality
89c4c71 feat: implement ZoomableImage component for enhanced image viewing experience
8aa3c08 refactor: replace 'SPV' with 'supervisor' for consistency across the application
981ea49 docs: add To-Be BPMN diagram in SVG and PNG formats
31485ee docs: add system architecture diagrams and documentation build script
b7baee9 feat: add product requirements document and update build script
1c81700 refactor: rebrand application to Kato Report and update theme color tokens
1302bf5 style: change hero links layout to vertical column in Home page
5192350 chore: update Open Graph image version and alt text metadata
defb699 feat: fail over to the next API key when one is rate-limited

Dokumentasi teknis · Kato Report · branch `staging` · sumber: `staging/docs/dokumentasi-teknis.html`